Pocket SDR C Library API Reference

ver.0.21 2026-10-21

Table of Contents

- Overview
- Constants and Macros
- Key Structures
- sdr_cmn.c - Common Utility Functions
- sdr_usb.c - USB Device Functions
- sdr_dev.c - Pocket SDR FE Device Functions
- sdr_sdev.c - SoapySDR Device Functions
- sdr_conf.c - SDR Device Configuration Functions
- sdr_func.c - Common SDR Functions
- sdr_code.c - GNSS Code Functions
- sdr_ch.c - Receiver Channel Functions
- sdr_nav.c - Navigation Data Decoder Functions
- sdr_fec.c - Forward Error Correction Functions
- sdr_ldpc.c - LDPC Decoder Functions
- sdr_nb_ldpc.c - Non-Binary LDPC Decoder Functions
- sdr_pvt.c - Position/Velocity/Time Functions
- sdr_array.c - Antenna Array Functions
- sdr_rcv.c - SDR Receiver Functions
- sdr_web.c - Web UI Server Functions

Overview

This API reference describes the **Pocket SDR** C library (`libsdr`). The library provides the building blocks for a software-defined GNSS receiver: USB / SoapySDR / file front-ends, IF buffer management, GNSS code generation and acquisition / tracking, navigation message decoding (FEC, LDPC, NB-LDPC), PVT (single-point positioning, observation/RTCM3 output), and antenna-array calibration and beam-forming. All entry points are declared in the single public header `pocket_sdr.h` .

- Scope and capabilities
  - Front-end I/O: Pocket SDR FE 2/4/8CH (Cypress EZ-USB), generic SoapySDR devices, IF data files, and TCP streams.
  - IF data processing: real (I) and complex (IQ) sampling, bit-packed RAW8 / RAW16 / RAW16I / RAW32 formats, fs/4 real-to-complex mixing, optional FIR LPF.
  - Acquisition and tracking: FFT-based code search with optional Doppler assistance and extended integration, fine Doppler estimation, multi-correlator DLL/PLL, noise-subtracted C/N0, Costas tracking, and coherent PLL updates for synchronized pilot signals.
  - Navigation decoding: per-system frame sync, convolutional / Reed-Solomon / LDPC / NB-LDPC decoders.
  - PVT: RTKLIB-based single-point positioning with antenna offsets and DCB; outputs include NMEA, RTCM3 obs/nav, and IF data log streams.
  - Antenna array: per-epoch EKF calibration of attitude (roll/pitch/yaw) and per-RF-CH hardware delays; beam-forming with quantized weights and up to 8 array channels.
- Conventions
  - Units: SI (m, s, Hz, sps). Angles are radians unless explicitly noted as degrees. Carrier phase observations are in cycles; pseudoranges in meters.
  - Time scales: GPST (`gtime_t` from RTKLIB) for navigation; UTC for receiver wall clock.
  - Frames: ENU for sky-plot directions; antenna positions are in body frame (X right, Y forward, Z up). Yaw is mathematical CCW from +X seen from +Z.
  - Memory: All `sdr_malloc()` allocations abort on failure (no NULL is returned). Buffers passed to APIs must remain valid for the call lifetime.
  - Threading: A receiver runs an IF data thread plus per-channel tracking threads. Public state queries (`sdr_rcv_*_stat`) take an internal mutex; concurrent `sdr_rcv_*` calls from the GUI thread are safe.
- Using the library
  - Include `pocket_sdr.h` and link `libsdr.a` (or `libsdr.so`). Initialize the library once with `sdr_func_init()` . Open a receiver via `sdr_rcv_open_dev` / `sdr_rcv_open_sdev` / `sdr_rcv_open_file` , then close with `sdr_rcv_close` .
  - For low-level use (custom front-ends), build the IF buffer / channel state directly with `sdr_buff_new` , `sdr_ch_new` , etc.

Constants and Macros

- **Library identity**
  - `SDR_LIB_NAME` = "Pocket SDR"
  - `SDR_LIB_VER` = library version string (e.g., "0.16")

- **Channel and buffer limits**
  - `SDR_MAX_RFCH` (8): maximum number of RF channels.
  - `SDR_MAX_ARCH` (8): maximum number of antenna-array channels.
  - `SDR_MAX_BUFF` (`SDR_MAX_RFCH+SDR_MAX_ARCH`): maximum number of IF data buffers.
  - `SDR_MAX_NPRN` (256): maximum PRNs per signal.
  - `SDR_MAX_NCH` (1500): maximum number of receiver channels.
  - `SDR_MAX_NSYM` (2000): maximum nav symbol buffer length.
  - `SDR_MAX_DATA` (4096): maximum nav data buffer length.
  - `SDR_N_CORRX` (101): number of additional (extra) correlators.
  - `SDR_MAX_CORR` (`6+SDR_N_CORRX`): total correlators per channel.
  - `SDR_N_HIST` (5000): P-correlator history length.
  - `SDR_N_CODES` (10): resampled code bank size.

- **Numeric scales**
  - `SDR_CSCALE` (≈ 1/11.2): carrier scale; keeps `max(IQ)·sqrt(2)/scale ≤ 127`.
  - `SDR_CYC` (1e-3 s): IF data processing cycle.
  - `PI` (3.14159265358979).

- **Device types** (`sdr_rcv_t.dev`)
  - `SDR_DEV_FILE` (1), `SDR_DEV_USB` (2), `SDR_DEV_STR` (3), `SDR_DEV_SOAPY` (4).

- **USB identifiers**
  - `SDR_DEV_NAME` = "Pocket SDR".
  - `SDR_DEV_VID` = 0x04B4 (Cypress).
  - `SDR_DEV_PID1` = 0x1004 (EZ-USB FX2LP), `SDR_DEV_PID2` = 0x00F1 (EZ-USB FX3).
  - `SDR_DEV_IF` = 0, `SDR_DEV_EP` = 0x86.

- **USB vendor requests**
  - `SDR_VR_STAT` (0x40): get status.
  - `SDR_VR_REG_READ` (0x41) / `SDR_VR_REG_WRITE` (0x42): MAX2771 register access.
  - `SDR_VR_START` (0x44) / `SDR_VR_STOP` (0x45): bulk transfer control.
  - `SDR_VR_RESET` (0x46): device reset.

    - `SDR_VR_SAVE` (0x47): persist settings.

- **IF data formats** (`sdr_rcv_t.fmt`)
    - `SDR_FMT_INT8` (1): int8 real (I).
    - `SDR_FMT_INT8X2` (2): int8×2 complex with Q-polarity flipped.
    - `SDR_FMT_RAW8` (3): packed 8-bit raw, 2 RF CH.
    - `SDR_FMT_RAW16` (4): packed 16-bit raw, 4 RF CH.
    - `SDR_FMT_RAW16I` (5): packed 16-bit raw, 8 RF CH (Spider SDR FE).
    - `SDR_FMT_RAW32` (6): packed 32-bit raw, 8 RF CH (Pocket SDR FE 8CH).
    - `SDR_FMT_CS8` (7): int8×2 complex.
    - `SDR_FMT_CS16` (8): int16×2 complex.

- **Channel states** (`sdr_ch_t.state`)
    - `SDR_STATE_IDLE` (1), `SDR_STATE_SRCH` (2), `SDR_STATE_LOCK` (3).

- **FFT direction**
    - `SDR_FFT_FORWARD` (0), `SDR_FFT_BACKWARD` (1).

- **Array calibration modes** (`sdr_array_t.calib_mode`)
    - `SDR_CALIB_BOTH` (0): estimate attitude + biases.
    - `SDR_CALIB_BIAS` (1): estimate biases only (rpy held at 0).
    - `SDR_CALIB_RPY` (2): estimate attitude only (biases held).

- **Packed complex helpers** (4-bit I + 4-bit Q in a single byte)
    - `SDR_CPX8(re, im)` : pack signed 4-bit I/Q into `sdr_cpx8_t` .
    - `SDR_CPX8_I(x)` : extract signed 4-bit I (sign-extended to int8).
    - `SDR_CPX8_Q(x)` : extract signed 4-bit Q (sign-extended to int8).

Key Structures

- `sdr_cpx8_t` (uint8_t alias)
  - Packed 4-bit I + 4-bit Q complex sample. Use `SDR_CPX8` / `SDR_CPX8_I` / `SDR_CPX8_Q` to access.

- `sdr_cpx16_t`
  - Fields: `int8_t I, Q`. 16-bit (8+8) signed complex sample.

- `sdr_cpx_t` (float[2])
  - Single-precision complex `{Re, Im}` for FFT and correlator outputs.

- `sdr_lpf_t` (opaque)
  - 9-tap symmetric FIR (Hamming-windowed sinc) low-pass filter state. Construct with `sdr_lpf_new`, apply with `sdr_lpf_apply`, free with `sdr_lpf_free`.

- `sdr_thread_t` / `sdr_mutex_t`
  - Thin platform abstractions over Win32 SRWLOCK / pthread. `SDR_MUTEX_INIT` provides a static initializer.

- `sdr_usb_t`
  - USB device handle (Cypress CyAPI on Windows; libusb-1.0 on Linux/macOS) with up to `SDR_MAX_UBUFF` outstanding transfers.

- `sdr_dev_t` (Pocket SDR FE)
  - USB device wrapper plus a ring buffer (`buff`, `rp`, `wp`) and a USB-event-handler thread/mutex.

- `sdr_sdev_t` (SoapySDR)
  - SoapySDR device/stream wrappers with a ring buffer, sample size, sample rate, reading thread/mutex.

- `sdr_acq_t`
  - Acquisition state: code FFT, Doppler bin array, external Doppler assist value plus explicit validity and assisted-bin count (`fd_ext_valid`, `fd_ext_n`), accumulated correlation power, and accumulation count.

- `sdr_trk_t`

- Tracking state: correlator positions and complex outputs; P history (`SDR_N_HIST`); secondary code sync/polarity; L6 CSK code-shift reference (`csk_ref`); phase/code error; DLL/PLL accumulators; prompt/noise and PLI power sums (`sumP`, `sumN`, `sumPs`, `sumD`); coherent pilot prompt state (`Cs`, `coh_n`); resampled real/complex code banks and the chip-domain extended-code FFT for L6 CSK detection.

- `sdr_nav_t`
  - Navigation decoder state: symbol/frame sync flags, polarity, error count, sequence/type/status, code offset (CSK), soft-symbol/data buffers, subframe lock times, OK/error counters. The navigation symbol buffer stores binary soft symbols in the range 0 to 255 except for L6 CSK paths, which store CSK symbol numbers.

- `sdr_ch_t`
  - Receiver baseband channel: identifier, code references, carrier/code/Doppler frequencies, C/N0 and PLI state, Costas and pilot classification flags, lock/loss counts, week/TOW, observation index, plus owned `acq` / `trk` / `nav` substructures.

- `sdr_buff_t`
  - IF data buffer: `data` array of `sdr_cpx8_t`, sampling type `IQ` (1=I, 2=IQ), and length `N`.

- `sdr_stats_t`
  - IF data statistics: std-dev, data rate (MB/s), cumulative size (MB), buffer usage (%), running accumulators for 8-bit and CS16 streams, sample count.

- `sdr_ch_th_t`
  - Channel thread state: run flag, owning `sdr_ch_t`, IF buffer read pointer, back-pointer to receiver, OS thread handle.

- `sdr_pvt_t`
  - PVT state: epoch and cycle index, receiver sync mode (`SDR_SYNC_UNKNOWN` / `SDR_SYNC_COARSE` / `SDR_SYNC_FINE`), satellite count, RTKLIB `obs_t` / `nav_t` / `sol_t` / `ssat_t` / `rtcm_t`, latency, sol/obs/nav counters, back-pointer to receiver, and the latest FIX position / receiver clock bias / receiver clock drift rate used by fast acquisition and persisted in `.pocket_navdata.csv`.

- `sdr_rfch_t`
  - Per-RF-CH configuration: LO frequency `fo`, sampling type `IQ`, sample bit width, optional `sdr_lpf_t`, and a raw→IF lookup table `LUT`.

- `sdr_arch_t`

- Per-array-CH beam state: azimuth/elevation, scale (`<= 0` disables this beam), quantized weight buffer `w[SDR_MAX_RFCH*2]` of int16 Q8 values `[Re_0, Im_0, Re_1, Im_1, ...]`, and a per-RF-CH `LUT[SDR_MAX_RFCH][256]` of `sdr_cpx64_t` precomputed by `sdr_arch_set_beam()` so the IF combine path is a pure LUT-add per sample.

- `sdr_array_t`
  - Antenna array state: frequency index, RF-CH count (`nrfch`), per-element enable flags and body-frame positions, calibration run flag, calibration mode (`SDR_CALIB_*`), epoch count, EKF state `x[3+SDR_MAX_RFCH]` = `{roll, pitch, yaw, bias_0..bias_{nrfch-1}}` (with `bias_0` held at 0 as the reference tie-down) and covariance `P`, RMS. Self-contained — no back-pointer to receiver. The number of array channels (`narch`) and per-array-CH beam states (`arch[]`) live in `sdr_rcv_t`.

- `sdr_web_cfg_t`
  - Receiver configuration held by the Web UI server and persisted in its settings file: input source (`inp` 0 = Pocket SDR FE, 1 = IF data, 2 = SoapySDR), IF data path / format / sampling rate, per-RF-CH LO frequencies, sampling types, sample bits and digital LPF bandwidths, replay time offset and scale, fast acquisition and RF CH separation flags, USB bus/port, device configuration file with its enable flag, RF frontend device options, SoapySDR driver, signal entries (`sig[]`, `prn[]`), output stream types and paths, receiver log type mask, RF CH assignments, receiver options, FFTW wisdom path and array element positions.

- `sdr_web_t` (opaque)
  - Web UI server state: listening socket, client table with per-client topic subscriptions, receiver pointer and its mutex, receiver log ring, and the configuration (`sdr_web_cfg_t`) with its settings file path.

- `sdr_rcv_t`
  - SDR receiver state: run flag, device type/pointer, IF data format / sampling rate / buffer length / channel counts (`nch`, `nrfch`, `narch`), current search channel, IF cycle count, per-RF-CH config (`rfch[]`), per-array-CH beam state (`arch[]`), per-CH IF buffers (RF + array), channel threads, optional `sdr_array_t`, `sdr_pvt_t`, IF data statistics, output streams `strs[SDR_MAX_STR]` with per-slot types `str_type[SDR_MAX_STR]` (`SDR_STR_???`), receiver start time (UTC), file replay scale, options string, fast-acquisition flag, IF data thread, and mutex.

sdr_cmn.c - Common Utility Functions

Overview

Process-wide utilities: zeroed-and-checked memory allocation, monotonic time, thread/mutex wrappers, and the library identity strings.

API Functions

`void *sdr_malloc(size_t size)`

- **Description**: Allocate `size` bytes initialized to zero.
- **Arguments**:
  - size: number of bytes
- **Return**: pointer to allocated buffer
- **Notes**:
  - Aborts the process on allocation failure (never returns NULL); callers must not add NULL checks.

`void sdr_free(void *p)`

- **Description**: Free a buffer obtained via `sdr_malloc()`.
- **Arguments**:
  - p: pointer (NULL is safe; treated as no-op)
- **Return**: none

`void sdr_get_time(double *t)`

- **Description**: Get current wall-clock time as `t[0..5]` = year/month/day/hour/minute/second (UTC).
- **Arguments**:
  - t: 6-element output array
- **Return**: none

`uint32_t sdr_get_tick(void)`

- **Description**: Monotonic millisecond tick (wraps every ~49.7 days).
- **Return**: tick value

`void sdr_sleep_msec(int msec)`

- **Description**: Sleep for `msec` milliseconds.
- **Arguments**:
  - msec: milliseconds (≤ 0: returns immediately)

`const char *sdr_get_name(void)`

- **Description**: Library name (`SDR_LIB_NAME`).
- **Return**: pointer to a static string

`const char *sdr_get_ver(void)`

- **Description**: Library version (`SDR_LIB_VER`).
- **Return**: pointer to a static string

`int sdr_thread_create(sdr_thread_t *thread, void *(func)(void *), void *arg)`

- **Description**: Start a new OS thread running `func(arg)` .
- **Arguments**:
  - thread: output thread handle
  - func: thread entry point
  - arg: opaque argument passed to `func`
- **Return**: 1 on success, 0 on error

`void sdr_thread_join(sdr_thread_t thread)`

- **Description**: Wait for `thread` to finish.
- **Arguments**:
  - thread: thread handle returned by `sdr_thread_create()`

`void sdr_mutex_init(sdr_mutex_t *mtx)`

- **Description**: Initialize a mutex (only required when `SDR_MUTEX_INIT` cannot be used).
- **Arguments**:
  - mtx: mutex pointer

`void sdr_mutex_lock(sdr_mutex_t *mtx)`

- **Description**: Acquire the mutex (blocking).

`void sdr_mutex_unlock(sdr_mutex_t *mtx)`

- **Description**: Release the mutex.

sdr_usb.c - USB Device Functions

Overview

Low-level USB transport for Pocket SDR FE devices. Wraps Cypress CyAPI (Windows) and libusb-1.0 (POSIX). Vendor requests are documented in the *Constants* section (`SDR_VR_*`).

API Functions

`sdr_usb_t *sdr_usb_open(int bus, int port, const uint16_t *vid, const uint16_t *pid, int n)`

- **Description**: Open the first USB device matching one of the listed VID/PID pairs.
- **Arguments**:
  - bus: USB bus number (-1 for any)
  - port: USB port number (-1 for any)
  - vid: array of `n` USB vendor IDs
  - pid: array of `n` USB product IDs
  - n: length of `vid` / `pid` arrays
- **Return**: USB device handle (NULL: not found / error)

`void sdr_usb_close(sdr_usb_t *usb)`

- **Description**: Close USB device.

`int sdr_usb_reset(sdr_usb_t *usb)`

- **Description**: Reset the USB device (CyAPI `Reset()` / `libusb_reset_device()`). The device re-enumerates, so it has to be closed and opened again after this call.
- **Return**: 1 on success, 0 on error

`int sdr_usb_req(sdr_usb_t *usb, int mode, uint8_t req, uint16_t val, uint8_t *data, int size)`

- **Description**: Issue a vendor-specific control transfer.
- **Arguments**:
  - usb: USB handle
  - mode: 0 for IN (device→host), 1 for OUT (host→device)
  - req: request code (one of `SDR_VR_*`)
  - val: wValue field
  - data: I/O buffer

    - size: buffer size in bytes
- **Return**: 1 on success, 0 on error

sdr_dev.c - Pocket SDR FE Device Functions

Overview

Higher-level USB front-end abstraction with bulk IF data transfer and optional MAX2771 control. Spawns a USB-event-handler thread that fills an internal ring buffer; consumers poll via `sdr_dev_read()`.

API Functions

`sdr_dev_t *sdr_dev_open(int bus, int port)`

- **Description**: Open and initialize a Pocket SDR FE device. A session killed without `sdr_dev_stop()` leaves the bulk transfer running; the FE 4CH/8CH reports it in the same status bit that `sdr_dev_get_info()` reads as the device type, so the FE would be taken for a Spider SDR. The open detects the state, resets the USB device and reopens it.
- **Arguments**:
    - bus: USB bus (-1 for any)
    - port: USB port (-1 for any)
- **Return**: device handle (NULL: error)

`void sdr_dev_close(sdr_dev_t *dev)`

- **Description**: Stop transfers, close USB, free buffers.

`int sdr_dev_start(sdr_dev_t *dev)`

- **Description**: Start IF data bulk transfers and the USB event thread.
- **Return**: 1 on success, 0 on error

`int sdr_dev_stop(sdr_dev_t *dev)`

- **Description**: Stop bulk transfers and join the event thread.
- **Return**: 1 on success, 0 on error

`int sdr_dev_read(sdr_dev_t *dev, uint8_t *buff, int size)`

- **Description**: Copy up to `size` bytes of IF data from the internal ring buffer.

- **Arguments**:
  - dev: device handle
  - buff: caller-supplied output buffer
  - size: byte count requested
- **Return**: number of bytes actually copied (0 if empty)

`int sdr_dev_get_info(sdr_dev_t *dev, int *fmt, double *fs, double *fo, int *IQ, int *bits)`

- **Description**: Read MAX2771 / FX2LP-FX3 configuration into out parameters.
- **Arguments**:
  - dev: device
  - fmt: IF data format (`SDR_FMT_*`)
  - fs: sampling rate (sps)
  - fo: per-RF-CH LO frequency array (length ≥ `SDR_MAX_RFCH`)
  - IQ: per-RF-CH sampling type (1=I, 2=IQ)
  - bits: per-RF-CH bit width
- **Return**: 1 on success, 0 on error

`int sdr_dev_get_gain(sdr_dev_t *dev, int ch)`

- **Description**: Read PGA gain of RF channel `ch` (1-indexed).
- **Return**: gain (dB) or negative on error

`int sdr_dev_set_gain(sdr_dev_t *dev, int ch, int gain)`

- **Description**: Set PGA gain of RF channel `ch` (1-indexed) to `gain` dB.
- **Return**: 1 on success, 0 on error

`int sdr_dev_get_filt(sdr_dev_t *dev, int ch, double *bw, double *freq, int *order)`

- **Description**: Read MAX2771 IF filter parameters of RF channel `ch` .
- **Arguments**:
  - bw: filter bandwidth (Hz, output)
  - freq: filter center frequency (Hz, output)
  - order: filter order (output)
- **Return**: 1 on success, 0 on error

`int sdr_dev_set_filt(sdr_dev_t *dev, int ch, double bw, double freq, int order)`

- **Description**: Set MAX2771 IF filter parameters of RF channel `ch` .

- **Return**: 1 on success, 0 on error

sdr_sdev.c - SoapySDR Device Functions

Overview

Generic SDR front-end via SoapySDR. Compiled in when `-DSOAPYSDR` is set. Supports any device whose driver advertises a contiguous CS8 / CS16 stream.

API Functions

`int sdr_sdev_list(void)`

- **Description**: Print available SoapySDR devices to stderr.
- **Return**: number of devices found

`sdr_sdev_t *sdr_sdev_open(const char *driver, int fmt, double rate, const double *freq, const char *opt)`

- **Description**: Open one or more SoapySDR devices as a single multi-channel device. With multiple devices the external clock and time sources are selected and the streams are started at a common time latched by a 1PPS, so that the RF channels are aligned in samples, unless `-NO_SYNC` is given.
- **Arguments**:
  - driver: SoapySDR driver string (e.g., "lime", "rtlsdr"), or device specs separated by `;`. A spec may be a SoapySDR key-value string, where the Pocket SDR extension key `nchan=<n>` selects the number of RX channels of the device (default 1). The total number of RF channels has to be `SDR_MAX_RFCH` or less.
    - `"driver=uhd,serial=1;driver=uhd,serial=2"`: 2 devices of 1 CH each
    - `"driver=uhd,serial=1,nchan=2"`: 1 device of 2 CHs
  - fmt: IF data format (`SDR_FMT_CS8` or `SDR_FMT_CS16`)
  - rate: sampling rate (sps)
  - freq: center frequency for each RF CH (Hz) (freq[i] ≤ 0: same as freq[0])
  - opt: device options separated by spaces
    - `-BW=<bw>`: analog filter bandwidth (MHz) (omitted: driver default)
    - `-GAIN=<gain>`: front-end gain (dB) (omitted: AGC)
    - `-EXT_CLOCK`: select the external clock source
    - `-NO_SYNC`: no device time sync by 1PPS. The devices run on their own clocks and the RF channels of different devices are NOT aligned in samples (for non-coherent uses such as a band per device)

    - `-GPIO=<val>`: output `<val>` (0-255) to the GPIO pins (USRP: front-panel GPIO bank `FP0`; other drivers: the first GPIO bank). The pins are set to outputs at the device open, since UHD resets them at every open.
- **Return**: device handle (NULL: error). The number of RF channels opened is in the `nch` member.

`void sdr_sdev_close(sdr_sdev_t *sdev)`

- **Description**: Close SoapySDR device.

`int sdr_sdev_start(sdr_sdev_t *sdev)`

- **Description**: Activate the streams and start the reading threads. With the device time sync, all the streams are started at a common time latched by a 1PPS. A multi-channel stream of a device is started at a time of the device in any case, as UHD requires it to align the channels.
- **Return**: 1 on success, 0 on error

`int sdr_sdev_stop(sdr_sdev_t *sdev)`

- **Description**: Deactivate the stream and join the reading thread.
- **Return**: 1 on success, 0 on error

`int sdr_sdev_read(sdr_sdev_t *sdev, uint8_t **buff, int size)`

- **Description**: Copy `size` bytes of IF data of each RF CH from the internal ring buffers into `buff[0]` ... `buff[nch-1]`. All the RF channels are read at the same sample position, so the data are aligned in samples.
- **Arguments**:
    - buff: IF data buffer of each RF CH (`buff[nch]`)
    - size: IF data buffer size of a RF CH (bytes)
- **Return**: number of bytes copied for a RF CH (0: no data, -1: error)

`int sdr_sdev_sensor(sdr_sdev_t *sdev, int ch, const char *name, double *value)`

- **Description**: Read a sensor of a RF channel, such as `temp`, `rssi` or `lo_locked` of a USRP. If the name is not a RF channel sensor, a sensor of the device (mboard) of the RF channel, such as `ref_locked`, is read instead. The sensor names are as defined by the device driver. A boolean sensor is read as 1 or 0.
- **Arguments**:
    - ch: RF channel index (0 to nch-1)
    - name: sensor name
    - value: sensor value

- **Return**: 1 on success, 0 if the device has no such sensor

sdr_conf.c - SDR Device Configuration Functions

Overview

Read and write Pocket SDR FE register configuration files (MAX2771 / FX2LP / FX3 register dumps).

API Functions

`int sdr_conf_read(sdr_dev_t *dev, const char *file, int opt)`

- **Description**: Apply registers from `file` to the device.
- **Arguments**:
  - dev: device handle
  - file: configuration file path
  - opt: option flags (reserved; pass 0)
- **Return**: 1 on success, 0 on error

`int sdr_conf_write(sdr_dev_t *dev, const char *file, int opt)`

- **Description**: Read the device's current registers and write them to `file`.
- **Return**: 1 on success, 0 on error

sdr_func.c - Common SDR Functions

Overview

General DSP and bookkeeping helpers: complex/FFT primitives, IF buffer / file I/O, code search and tracking correlators (scalar + AVX2 / AVX-512 paths), Doppler bin generation, PSD utilities, log streams, packed-bit accumulators, and the digital LPF.

API Functions

`void sdr_func_init(const char *file)`

- **Description**: Library-wide initialization (FFTW wisdom, tables, RNG seed). Must be called once before any other SDR functions.
- **Arguments**:
  - file: FFTW wisdom file path (NULL or empty: skip)

`sdr_cpx_t *sdr_cpx_malloc(int N)`

- **Description**: Allocate `N`-element single-precision complex array (FFTW-aligned).
- **Return**: pointer

`void sdr_cpx_free(sdr_cpx_t *cpx)`

- **Description**: Free a buffer from `sdr_cpx_malloc()`.

`float sdr_cpx_abs(sdr_cpx_t cpx)`

- **Description**: Absolute value of a single complex sample.
- **Return**: |cpx|

`void sdr_cpx_mul(const sdr_cpx_t *a, const sdr_cpx_t *b, int N, float s, sdr_cpx_t *c)`

- **Description**: Element-wise complex multiply with scale: `c[i] = s * a[i] * b[i]`.

`int sdr_cpx_fft(sdr_cpx_t *cpx1, int N, int dir, sdr_cpx_t *cpx2)`

- **Description**: N-point FFT via FFTW3 or PocketFFT.
- **Arguments**:
  - cpx1: input array (length N)

    - N: FFT length
    - dir: `SDR_FFT_FORWARD` or `SDR_FFT_BACKWARD`
    - cpx2: output array (length N)
- **Return**: 1 on success, 0 on error

`sdr_buff_t *sdr_buff_new(int N, int IQ)`

- **Description**: Allocate an IF data buffer of length `N` samples with sampling type `IQ` (1=I, 2=IQ).
- **Return**: buffer pointer (with `data` set to `sdr_malloc`'d storage)

`void sdr_buff_free(sdr_buff_t *buff)`

- **Description**: Free an IF data buffer.

`sdr_buff_t *sdr_read_data(const char *file, double fs, int IQ, double T, double toff)`

- **Description**: Read up to `T` seconds of IF data from a file starting at offset `toff` seconds.
- **Arguments**:
    - file: IF data file path
    - fs: sampling rate (sps)
    - IQ: sampling type (1=I, 2=IQ)
    - T: duration to read (s)
    - toff: file offset (s)
- **Return**: filled `sdr_buff_t` (NULL: error)

`int sdr_tag_write(const char *file, const char *prog, gtime_t time, int fmt, int nch, double fs, const double *fo, const int *IQ, const int *bits)`

- **Description**: Write a `<file>.tag` companion file describing IF data parameters. `nch` (≤ 0: as defined by `fmt`) is written as `NCH`. If `nch` exceeds the number of RF channels defined by `fmt`, the IF data file is a sequence of channel-planar blocks of `SDR_CYC` each, and `T_CYC` is also written so that a replay with a different processing cycle is rejected.
- **Return**: 1 on success, 0 on error

`int sdr_tag_read(const char *file, char *prog, gtime_t *time, int *fmt, int *nch, double *fs, double *fo, int *IQ, int *bits)`

- **Description**: Read a `<file>.tag` companion file. Output pointers may be NULL to skip. `nch` falls back to the number of RF channels defined by `fmt` when the tag file has no `NCH`, so tag files written by earlier versions are read as before.

- **Return**: 1 on success, 0 on no readable tag file, -1 if the IF data cannot be used (`T_CYC` does not match `SDR_CYC`)

`void sdr_search_code(const sdr_cpx_t *code_fft, double T, const sdr_buff_t *buff, int ix, int N, double fs, double fi, const float *fds, int len_fds, float *P)`

- **Description**: Run FFT-based code/Doppler search over the precomputed conjugate-spectrum `code_fft` and the IF buffer slice `[ix, ix+N)` . Accumulates `|R(f_d, τ)|²` into `P` (`len_fds × N` row-major).

`float sdr_corr_max(const float *P, int N, int Nmax, int M, double T, int *ix)`

- **Description**: Find the peak of the 2D power matrix `P` and return its peak-to-mean ratio (PMR).
- **Arguments**:
  - P: power matrix
  - N: code dimension length
  - Nmax: number of code samples to consider
  - M: Doppler dimension length
  - T: code period (s)
  - ix: 2-element array; `ix[0]` = Doppler index, `ix[1]` = code-offset sample
- **Return**: PMR

`double sdr_fine_dop(const float *P, int N, const float *fds, int len_fds, const int *ix)`

- **Description**: Estimate Doppler with sub-bin resolution by parabolic fit around `ix[0]` .
- **Return**: Doppler frequency (Hz)

`double sdr_shift_freq(const char *sig, int fcn, double fi)`

- **Description**: Apply a signal-specific carrier shift (e.g., GLONASS FDMA channel offset) to the nominal IF.
- **Arguments**:
  - sig: signal name (e.g., "L1CA", "G1CA")
  - fcn: GLONASS frequency channel number
  - fi: nominal IF (Hz)
- **Return**: shifted IF (Hz)

`float *sdr_dop_bins(double T, float dop, float max_dop, int *len_fds)`

- **Description**: Generate Doppler bins centered at `dop` spaced by ≈ 1/(2T) up to ± `max_dop` .
- **Arguments**:

    - T: code period (s)
    - dop: center Doppler (Hz)
    - max_dop: half-range (Hz)
    - len_fds: output length
- **Return**: heap-allocated float array (free with `sdr_free()`)

`void sdr_corr_std(const sdr_buff_t *buff, int ix, int N, double fs, double fc, double phi, const int8_t *code, const int32_t *code_sum, const int8_t *code_Q, const int32_t *code_sum_Q, int scale, double coff, const double *pos, int n, int pol, sdr_cpx_t *corr, sdr_cpx_t *C)`

- **Description**: Standard time-domain correlator with fused carrier wipeoff. Mixes the carrier (`fc`, `phi`) and correlates against the code replicas in a single chunked pass over `buff[ix..ix+N)`, without an intermediate baseband buffer. `code` is the resampled code bank (`N x SDR_N_CODES`, `int8_t` values, I=Q implied) and `code_sum` its prefix sums (`(N+1) x SDR_N_CODES`, used to remove the +128 data bias of the uint8 x int8 inner products). `code_Q` / `code_sum_Q` are the imaginary code bank for complex replicas (NULL for a real code); with them the correlation is `sum(IQ * conj(code + j*code_Q))`. `scale` is the code amplitude scale (1 for -1/0/+1 codes); correlations are normalized by it. Writes `n` correlations to `corr` and the pre/post code-wrap prompt correlations to `C[0..1]` (bit-transition handling). `pol` selects the code polarity handling over the wrap-around: 0 detects a data-bit flip (normal tracking), +1/-1 assumes fixed polarity (symbol-aligned circular window, e.g. L6 CSK).

`void sdr_corr_std2(const sdr_buff_t *buff, int ix, int N, double fs, double fc, double phi, const int8_t *code, const int32_t *code_sum, const int8_t *code_Q, const int32_t *code_sum_Q, const int8_t *code2, const int32_t *code2_sum, const int8_t *code2_Q, const int32_t *code2_sum_Q, int scale, double coff, const double *pos, int n, int pol, sdr_cpx_t *corr, sdr_cpx_t *C)`

- **Description**: `sdr_corr_std()` with separate code banks for the part after the code wrap-around: `code2*` (NULL = same as `code*`) allow a replica whose sideband combination sign differs across the wrap-around, which `pol` cannot express (E5 AltBOC a/b combining).

`void sdr_corr_std_cpx(const sdr_cpx_t *buff, int len_buff, int ix, int N, double fs, double fc, double phi, const float *code, const double *pos, int n, sdr_cpx_t *corr)`

- **Description**: Time-domain correlator that mixes carrier (`fc`, `phi`) on the fly while accumulating into `corr`.

`void sdr_corr_fft(const sdr_buff_t *buff, int ix, int N, double fs, double fc, double phi, const sdr_cpx_t *code_fft, sdr_cpx_t *corr)`

- **Description**: Frequency-domain correlation with fused carrier wipeoff. Mixes the carrier (`fc` , `phi`) on `buff[ix..ix+N)` directly into the FFT input, then forward FFT, multiply by `code_fft` , inverse FFT into `corr` . Uses a per-thread persistent scratch buffer (no per-call allocation).

`void sdr_corr_fft_cpx(const sdr_cpx_t *buff, int len_buff, int ix, int N, double fs, double fc, double phi, const sdr_cpx_t *code_fft, sdr_cpx_t *corr)`

- **Description**: Same as above but mixes carrier from a complex IF buffer.

`void sdr_mix_carr(const sdr_buff_t *buff, int ix, int N, double fs, double fc, double phi, sdr_cpx16_t *IQ)`

- **Description**: Wipe-off carrier from `buff[ix..ix+N)` using `cos/sin(2π·fc·t + phi)` and write 8-bit complex baseband to `IQ` .

`void sdr_bin_csk(const sdr_cpx16_t *IQ, int N, int len_code, int slot, double coff, sdr_cpx_t *bins)`

- **Description**: Bin carrier-mixed IF data to code chips for L6 CSK decoding. Accumulates samples of one of the two TDM sub-chip slots per code chip (L6D: `slot=0` , L6E: `slot=1`) into `bins` (`len_code/2` chips). `coff` is the fractional code offset in samples.

`void sdr_psd_cpx(const sdr_cpx_t *buff, int len_buff, int N, double fs, int IQ, float *psd)`

- **Description**: Compute a Welch-style PSD over `len_buff` samples with FFT length `N` .
- **Arguments**:
  - psd: output array of length `N` (IQ=2) or `N/2` (IQ=1) in dB/Hz

`stream_t *sdr_str_open(const char *path)`

- **Description**: Open an RTKLIB output stream (file/serial/TCP/...).

`void sdr_str_close(stream_t *str)`

- **Description**: Close an RTKLIB output stream.

`int sdr_str_write(stream_t *str, uint8_t *data, int size)`

- **Description**: Write `size` bytes to `str` .
- **Return**: number of bytes written

`int sdr_log_open(const char *path)`

- **Description**: Open the library-owned log stream and register it as a log output.
- **Return**: 1 on success, 0 on error

`void sdr_log_close(void)`

- **Description**: Close the library-owned log stream opened by `sdr_log_open()`.

`void sdr_log_add_str(stream_t *str)`

- **Description**: Register an externally owned stream as a log output (up to `SDR_MAX_STR`; duplicates ignored). `sdr_log()` writes each record to every registered stream.

`void sdr_log_rm_str(stream_t *str)`

- **Description**: Unregister a stream registered by `sdr_log_add_str()`. The stream itself is not closed.

`void sdr_log_level(int level)`

- **Description**: Set the log verbosity level (higher = more verbose).

`void sdr_log_mask(const int *mask, int n)`

- **Description**: Set per-tag log filtering mask of length `n`.

`void sdr_log(int level, const char *msg, ...)`

- **Description**: Emit a printf-formatted message at `level`.

`int sdr_log_stat(void)`

- **Description**: Get the connection state of the first registered log stream (0 if none).

`int sdr_get_log(char *buff, int size)`

- **Description**: Read accumulated log lines into `buff` (NUL-terminated).
- **Return**: number of bytes written

`int sdr_parse_nums(const char *str, int *prns)`

- **Description**: Parse an integer list like `"1,3-5,7"` into `prns[]`.
- **Return**: number of values parsed

`void sdr_add_buff(void *buff, int len_buff, void *item, size_t size_item)`

- **Description**: Shift `buff` left by `size_item` bytes and append `item` at the tail (FIFO insert at end).

`void sdr_pack_bits(const uint8_t *data, int nbit, int nz, uint8_t *buff)`

- **Description**: Pack `nbit` data bits (one bit per byte) into `buff`, prepending `nz` zero pad bits.

`void sdr_unpack_bits(const uint8_t *data, int nbit, uint8_t *buff)`

- **Description**: Unpack `nbit` bits from packed `data` into `buff` (one bit per byte).

`void sdr_unpack_data(uint32_t data, int nbit, uint8_t *buff)`

- **Description**: Unpack the lower `nbit` bits of `data` (MSB-first) into `buff`.

`uint8_t sdr_xor_bits(uint32_t X)`

- **Description**: Population-count parity of `X` (XOR of all bits).
- **Return**: 0 or 1

`int sdr_gen_fftw_wisdom(const char *file, int N)`

- **Description**: Generate FFTW wisdom for FFT length `N` and save to `file`.
- **Return**: 1 on success, 0 on error

`sdr_lpf_t *sdr_lpf_new(double fc, double fs)`

- **Description**: Allocate a 9-tap Hamming-windowed FIR low-pass filter with one-sided cutoff `fc`.
- **Arguments**:
  - fc: cutoff frequency (Hz; must satisfy `0 < fc < fs/2`)
  - fs: sampling rate (Hz)
- **Return**: LPF instance (NULL on invalid arguments)

`void sdr_lpf_free(sdr_lpf_t *lpf)`

- **Description**: Free an LPF instance (NULL safe).

`void sdr_lpf_apply(sdr_lpf_t *lpf, sdr_cpx8_t *data, int N)`

- **Description**: Apply the LPF in place to `N` packed complex samples. Filter state is updated for streaming use.

sdr_code.c - GNSS Code Functions

Overview

GNSS spreading-code generators and resampling: GPS L1/L2/L5, Galileo E1/E5a/E5b/E6, BeiDou B1I/B1C/B2a/B2b/B3I, QZSS L1/L2/L5/L6, GLONASS L1/L2 (FDMA), NavIC L5/S, SBAS L1/L5. Includes secondary code lookup, resampling, and pre-FFT code spectra for fast acquisition.

API Functions

`int8_t *sdr_gen_code(const char *sig, int prn, int *N)`

- **Description**: Return a pointer to the primary spreading code for `(sig, prn)`. Length is written to `*N`.
- **Arguments**:
  - sig: signal name (e.g., "L1CA")
  - prn: PRN number
  - N: code length (chips, output)
- **Return**: pointer to internal code (read-only); NULL on invalid input

`int8_t *sdr_sec_code(const char *sig, int prn, int *N)`

- **Description**: Return the secondary code for `(sig, prn)` and its length in `*N`.
- **Return**: pointer to internal code (read-only); NULL if none

`double sdr_code_cyc(const char *sig)`

- **Description**: Primary-code period in seconds.

`int sdr_code_len(const char *sig)`

- **Description**: Primary-code length in chips.

`double sdr_sig_freq(const char *sig)`

- **Description**: Nominal RF carrier frequency for `sig` (Hz).

`void sdr_sat_id(const char *sig, int prn, char *sat)`

- **Description**: Compose RTKLIB satellite ID from signal/PRN (e.g., "G01", "E05").

`int sdr_sig_boc(const char *sig)`

- **Description**: Return 1 for `L1CD`, `L1CP`, `G1OCP`, `G2OCP`, `E1B`, `E1C`, `L1CB`, `B1CD`, `B1CP`, `I1SP`, `I1SD`, or `E5ABQ`; return 0 otherwise. This is a BOC/MBOC-family classification flag, not the BOC order.

`int sdr_sig_pilot(const char *sig)`

- **Description**: Return 1 for `L1CP`, `L5Q`, `L5SQ`, `G2OCP`, `G3OCP`, `E1C`, `E5AQ`, `E5ABQ`, `E5BQ`, `E6C`, `B1CP`, `B2AP`, or `I1SP`; return 0 otherwise. The channel tracker uses this classification to enable coherent prompt accumulation after secondary-code synchronization.

`int sdr_code_scale(const char *sig)`

- **Description**: Return the integer chip-amplitude scale used by the generated spreading code. Returns `SDR_CBOC_SCALE` for multi-level Galileo E1 CBOC codes and 1 for ordinary `-1/0/+1` codes.

`void sdr_res_code(const int8_t *code_I, const int8_t *code_Q, int len_code, double T, double coff, double fs, int N, int Nz, sdr_cpx16_t *code_res)`

- **Description**: Resample a spreading code at sample rate `fs` with code offset `coff`, length-`N` output and `Nz` zero-padding. Pass `code_Q = NULL` for real (BPSK) codes; non-NULL for complex (I+Q) codes.

`void sdr_gen_code_fft(const int8_t *code_I, const int8_t *code_Q, int len_code, double T, double coff, double fs, int N, int Nz, sdr_cpx_t *code_fft)`

- **Description**: Resample a code and produce its conjugate FFT (matched-filter spectrum) of length `N` for use with `sdr_search_code()`. `code_Q = NULL` for real codes.

sdr_ch.c - Receiver Channel Functions

Overview

Per-signal baseband channel: FFT acquisition with optional Doppler-assisted extended integration, fine Doppler, third-order PLL (Costas or synchronized-pilot coherent mode), second-order DLL, secondary-code sync, noise-subtracted C/N0/PLI loss detection, and lock counters. The IF data is consumed via `sdr_ch_update()`; correlator state is exposed via `sdr_ch_corr_stat()` / `sdr_ch_corr_hist()`.

API Functions

`sdr_ch_t *sdr_ch_new(const char *sig, int prn, double fs, double fi)`

- **Description**: Allocate and initialize a receiver channel for `(sig, prn)` at sampling rate `fs` and IF `fi`.
- **Return**: channel pointer (NULL on invalid signal)

`void sdr_ch_free(sdr_ch_t *ch)`

- **Description**: Free a receiver channel (along with its `acq` / `trk` / `nav` state).

`void sdr_ch_update(sdr_ch_t *ch, double time, const sdr_buff_t *buff, int ix)`

- **Description**: Run one update cycle: acquisition or tracking step on the IF buffer slice starting at `ix`.
- **Arguments**:
  - ch: channel
  - time: receiver time (s) for log/state stamping
  - buff: IF data buffer
  - ix: read pointer (samples)

`void sdr_ch_set_corr(sdr_ch_t *ch, int nposx, double width)`

- **Description**: Configure the extra correlators used for diagnostics: `nposx` positions (0.. `SDR_N_CORRX`) spanning a total width of `width` seconds (centered on the prompt).

`int sdr_ch_corr_stat(sdr_ch_t *ch, double *stat, double *pos, sdr_cpx_t *C, double *P, double *I)`

- **Description**: Snapshot the current correlator state.
- **Arguments**:
  - stat: scalar stats {state, fs, lock time (s), C/N0 (dB-Hz), code offset (ms), Doppler (Hz), base correlator count}
  - pos: correlator positions (samples)
  - C: complex correlator outputs
  - P: average correlator powers (`aveP`, length = correlator count)
  - I: data-wipe-off accumulator (`I·sign(IP)` averages, length = correlator count)
- **Return**: number of correlator positions (`npos + nposx`)

`int sdr_ch_corr_hist(sdr_ch_t *ch, double tspan, double *stat, sdr_cpx_t *P)`

- **Description**: Return P-correlator history covering the past `tspan` seconds.
- **Arguments**:
  - stat: scalar stats {receiver time (s), code period `T` (s)}
  - P: prompt-correlation history (most recent last)
- **Return**: number of samples returned

sdr_nav.c - Navigation Data Decoder Functions

Overview

Top-level navigation message decoders. Each call to `sdr_nav_decode()` consumes new soft symbols from the channel's tracking output, performs frame sync, runs the appropriate FEC, and emits decoded ephemerides / almanacs to the receiver's `nav_t`. Binary navigation soft symbols use `0` for a strong 0, `128` for the hard-decision boundary, and `255` for a strong 1. Hard-only parity, BCH, CRC, and table-matching paths derive hard bits locally.

API Functions

`sdr_nav_t *sdr_nav_new(void)`

- **Description**: Allocate a navigation decoder state (zero-initialized).

`void sdr_nav_free(sdr_nav_t *nav)`

- **Description**: Free the decoder state.

`void sdr_nav_init(sdr_nav_t *nav)`

- **Description**: Reset all sync flags and counters in `nav`.

`void sdr_nav_decode(sdr_ch_t *ch)`

- **Description**: Decode navigation soft symbols for the channel's signal type. Updates `ch->nav` and emits to the parent receiver's `nav_t` when a frame is complete.

sdr_fec.c - Forward Error Correction Functions

Overview

Convolutional decoder (rate-1/2, k=7, generator [171, 133]) and Reed-Solomon decoder used by GPS L5 / L2C / L6 and similar.

API Functions

`void sdr_decode_conv(const uint8_t *data, int N, uint8_t *dec_data)`

- **Description**: Viterbi decode `N` rate-1/2 soft symbols (`uint8_t` in the range 0 to 255, with 128 as the hard-decision boundary) into `N/2` hard bits.

`int sdr_decode_rs(uint8_t *syms)`

- **Description**: Decode an RS(255,223) codeword in place; returns the number of corrected errors or -1 on failure.

sdr_ldpc.c - LDPC Decoder Functions

Overview

Belief-propagation LDPC decoder for BeiDou B1C, B2a, B2b, GPS L1C, etc. The decoder interface is soft-decision input; callers that only have hard decisions should pass 0/255 symbols.

API Functions

`int sdr_decode_LDPC(const char *type, const uint8_t *syms, int N, uint8_t *syms_dec)`

- **Description**: Decode `N` binary LDPC soft symbols using the parity-check matrix selected by `type` (for example, "CNV2_SF2", "BCNV2", "IRNV1_SF2"). Input `syms` uses `uint8_t` soft values in the range 0 to 255, where 0 is a strong 0, 128 is the hard-decision boundary, and 255 is a strong 1. Output `syms_dec` contains decoded hard bits as 0 or 1, without parity bits.
- **Return**: number of corrected errors or -1 on decode failure

sdr_nb_ldpc.c - Non-Binary LDPC Decoder Functions

Overview

Non-binary LDPC decoder (used by some new GNSS signals). The parity-check matrix is supplied by the caller. The decoder interface is soft-decision input; callers that only have hard decisions should pass 0/255 symbols.

API Functions

`int sdr_decode_NB_LDPC(const uint8_t H_idx[][4], const uint8_t H_ele[][4], int m, int n, const uint8_t *syms, uint8_t *syms_dec)`

- **Description**: Decode a non-binary LDPC codeword with parity-check matrix described by `H_idx` / `H_ele` (m x n). Input `syms` contains `n * 6` binary soft symbols (`uint8_t` in the range 0 to 255, with 128 as the hard-decision boundary). Output `syms_dec` contains decoded hard bits as 0 or 1.
- **Return**: 1 on success, 0 on failure

sdr_pvt.c - Position/Velocity/Time Functions

Overview

Single-point positioning and observation aggregation built on RTKLIB. The receiver thread feeds observations and decoded nav data into the PVT state, then `sdr_pvt_udsol()` runs `pntpos()` and dispatches output streams (NMEA, RTCM3, $POS log).

API Functions

`sdr_pvt_t *sdr_pvt_new(sdr_rcv_t *rcv)`

- **Description**: Allocate a PVT state owned by `rcv`. `rcv->pvt` is set automatically inside `sdr_rcv_*` openers.

`void sdr_pvt_free(sdr_pvt_t *pvt)`

- **Description**: Free the PVT state.

`void sdr_pvt_udobs(sdr_pvt_t *pvt, int64_t ix, sdr_ch_t *ch)`

- **Description**: Update observation data for IF cycle `ix` from a tracking channel.

`void sdr_pvt_udnav(sdr_pvt_t *pvt, sdr_ch_t *ch)`

- **Description**: Update navigation data from a tracking channel that completed a frame.

`void sdr_pvt_udsol(sdr_pvt_t *pvt, int64_t ix)`

- **Description**: Run the per-epoch PVT solution: ambiguity check, sort obs, write logs/streams, call `pntpos()`, update `sol` / `ssat`, trigger array calibration step (`sdr_rcv_array_calib()`), and advance the next epoch time.

`void sdr_pvt_solstr(sdr_pvt_t *pvt, char *buff, int size)`

- **Description**: Format the latest PVT solution into a single-line text string.
- **Return**: none (writes up to `size` bytes including NUL)

sdr_array.c - Antenna Array Functions

Overview

Antenna-array calibration and beam-forming. Calibration uses a per-epoch EKF over single-difference carrier-phase residuals between RF channel 1 (reference) and the others. The EKF state is `{roll, pitch, yaw, bias_1..bias_n}` (bias_1 is held near zero by a rank-deficiency tie-down so the remaining biases are interpreted relative to CH1). Beam-forming weights are quantized int16 (Q8) per RF CH so they can be applied with integer math in the IF data path. Per-CH beam state lives in `sdr_rcv_t.arch[]`; the calibration EKF state lives in `sdr_array_t`.

Models and Algorithms

- Body-to-ENU rotation: `R = Rz(yaw)·Ry(pitch)·Rx(roll)` (X-Y-Z = right / forward / up).
- Calibration SD model: `λ·SD_L ≈ proj_calib − bias_calib`, where `proj_calib = sd_proj() = -e·R·b` (`e` = LOS unit vector in ENU, `b = b_k − b_0` body-frame baseline).
- Bias sign convention (important): `bias_calib = -bias_phys`. The EKF stores the **negation** of the physical per-channel cable delay (in meters); a positive physical delay on CH `k` produces a negative entry in the state vector.
- Beam-forming weight: `w_a = (scale · bit_gain) · exp(j · phi_a)` with `phi_a = -2π/λ · (proj_beam + bias_calib)` and `proj_beam = e_body · b = (R^T·e)·b = +e·R·b`. The two `proj` definitions deliberately differ in sign so the bias term is **added**, not subtracted, in the beam formula. Weights are quantized to int16 Q8 (`ARRAY_W_SCALE = 256`).
- Initialization: yaw grid search (15° step) plus iterative LSQ per epoch; subsequent calls run a standard EKF predict (random-walk) + update.

API Functions

`sdr_array_t *sdr_array_new(int nrfch, int freq)`

- **Description**: Allocate and initialize an antenna array. Initial `ant_pos` is all-zero and `ant_ena[0..nrfch-1]` is 1. The struct is self-contained — no back-pointer to a receiver — so the same constructor serves both the receiver-internal use (`sdr_rcv.c`) and standalone batch calibration (`pocket_calib`).
- **Arguments**:
  - nrfch: number of RF channels (1..`SDR_MAX_RFCH`)
  - freq: frequency index (0=L1, 1=L2, ...)
- **Return**: array pointer (never NULL — `sdr_malloc` aborts on failure)
- **Notes**:
  - Antenna geometry and calibration state are configured via `sdr_array_ant_pos()` / `sdr_array_set()` / `sdr_array_run()` after construction.

`void sdr_array_free(sdr_array_t *array)`

- **Description**: Free the array state (NULL-safe).

`int sdr_array_ant_pos(sdr_array_t *array, const double *ant_pos, const int *ant_ena)`

- **Description**: Update body-frame antenna positions and per-element enable flags.
- **Arguments**:
  - array: array state (uses `array->nrfch` for buffer sizes)
  - ant_pos: `array->nrfch * 3` doubles in row-major order
  - ant_ena: `array->nrfch` enable flags (`ant_ena[0]` must be 1, the call is rejected otherwise)
- **Return**: 1 on success, 0 when `ant_ena[0] == 0`

`int sdr_array_run(sdr_array_t *array, int run)`

- **Description**: Drive the EKF run flag.
- **Arguments**:
  - array: array state (must satisfy `array->nrfch >= 2`)
  - run: 0 stop, 1 start fresh (resets EKF state), 2 clear (resets state and stops)
- **Return**: 1 on success, 0 on error

`int sdr_array_set_mode(sdr_array_t *array, int mode)`

- **Description**: Select what the EKF estimates: `SDR_CALIB_BOTH` (rpy + bias), `SDR_CALIB_BIAS` (bias only, rpy clamped to 0), or `SDR_CALIB_RPY` (rpy only, bias frozen).
- **Return**: 1 on success, 0 on error

`int sdr_array_set(sdr_array_t *array, const double *rpy, const double *bias)`

- **Description**: Set calibration values into the EKF state and re-seed the covariance.
- **Arguments**:
  - array: array state (must satisfy `array->nrfch >= 2`)
  - rpy: attitude {roll, pitch, yaw} (3 doubles)
  - bias: per-RF-CH calibration biases (`array->nrfch` doubles)
- **Return**: 1 on success, 0 on error

`int sdr_array_stat(sdr_array_t *array, double *rpy, double *bias, double *rms, int *nep)`

- **Description**: Read the current calibration state. `bias[0]` is always returned as 0 (reference CH); `bias[1..nrfch-1]` come from the EKF state.

- **Return**: current calibration run flag (1 = running, 0 = stopped); 0 also on error.

`void sdr_array_calib(sdr_array_t *array, const obsd_t *obs, int nobs, const nav_t *nav, const double *rr)`

- **Description**: Per-epoch calibration driver. When the run flag is set and `obs` / `rr` are valid, runs one EKF init or update step and increments the epoch counter on success.

`int sdr_array_save(sdr_array_t *array, const char *file)`

- **Description**: Snapshot the current EKF state (rpy + per-CH biases) into `file` in the text format consumed by `sdr_array_load()` / `pocket_trk -opt`. Returns 0 (no write) when no calibration data has been produced yet.
- **Return**: 1 on success, 0 on error / nothing to save

`int sdr_array_load(sdr_array_t *array, const char *file)`

- **Description**: Load attitude and per-CH biases from `file` and inject them into the EKF state, re-seeding the covariance.
- **Return**: 1 on success, 0 on error

`int sdr_array_geom_save(const char *file, const double *ant_pos, int n)`

- **Description**: Write `n` body-frame antenna positions (`ant_pos`, row-major `n*3` doubles, m) to `file` as a text geometry file (one `x y z` per line). The first row must be `0 0 0` (CH1 reference).
- **Return**: 1 on success, 0 on error

`int sdr_array_geom_load(const char *file, double *ant_pos, int max_ant)`

- **Description**: Read up to `max_ant` antenna positions from `file` into `ant_pos` (row-major `max_ant*3`).
- **Return**: number of antenna positions read, 0 on error

`void sdr_arch_free(sdr_arch_t *arch)`

- **Description**: Reset a per-array-CH beam state in place (zero az/el/scale and clear weights). NULL-safe.

`void sdr_arch_set_beam(sdr_arch_t *arch, const sdr_rcv_t *rcv, double az, double el, double scale)`

- **Description**: Compute beam-forming weights for one array CH pointing in ENU direction (`az`, `el`) and install them into `arch`.
- **Arguments**:
  - arch: per-array-CH beam state to update
  - rcv: receiver (must have `rcv->array != NULL`)
  - az: beam azimuth (rad, from N CW)
  - el: beam elevation (rad, above horizon)
  - scale: per-CH weight magnitude (`<= 0` disables this beam — installs zero weights)
- **Notes**:
  - Reads the current rpy/bias and ant_pos from `rcv->array`. Caller is responsible for holding `rcv->mtx` when calling concurrently with `sdr_arch_combine()`.
  - Bit-range expansion gain is applied automatically based on the smallest `bits` among participating L1 RF CHs.

`void sdr_arch_get_beam(const sdr_arch_t *arch, double *az, double *el)`

- **Description**: Read the most recently set beam direction into `*az` / `*el`.

`void sdr_arch_combine(const sdr_arch_t *arch, const sdr_rcv_t *rcv, int base)`

- **Description**: Combine RF CH IF data into one array CH IF data buffer using the current weights. Reads `rcv->buff[0..nrfch-1]` and writes `rcv->buff[nrfch + m]` (where `m = arch - rcv->arch`), each starting at sample offset `base`. No-op when `arch->scale <= 0`.
- **Arguments**:
  - arch: per-array-CH beam state (selects the destination buffer by its offset within `rcv->arch[]`)
  - rcv: receiver
  - base: starting sample offset in each buffer
- **Return**: none
- **Notes**:
  - Caller-locked: the receiver loop calls this under `rcv->mtx` so beam updates are atomic relative to the integer combine.

sdr_rcv.c - SDR Receiver Functions

Overview

Top-level receiver lifecycle and state queries. A receiver wraps a front-end (USB / SoapySDR / file / stream), an IF data thread that demuxes raw samples into per-RF-CH `sdr_buff_t` s, multiple `sdr_ch_t` channel threads driving acquisition / tracking / nav decoding, an `sdr_pvt_t` for solutions, an optional `sdr_array_t`, and up to `SDR_MAX_STR` (= 8) output streams. Each stream slot is freely assigned one of the types NMEA PVT solutions, RTCM3 OBS+NAV, event log, or raw IF data log; the same type may be assigned to multiple slots (the message is encoded once and written to every stream of the type).

Receiver Options String

`sdr_rcv_open_*` and `sdr_rcv_new` take an `opt` string with space-separated tokens:

- `-RFCH <sig>:<ch>[{,|-}<ch>...]` — assign a signal to specific RF CH(s). Array CHs (`nrfch+1` .. `nrfch+narch`) can be assigned as well.
- `-LPF=<rfch>:<MHz>[,<rfch>:<MHz>...]` — enable digital LPF on RF CH(s) (two-sided BW in MHz). Real-sampling RF CHs (`IQ=1`) are auto-LPFed at 0.9·fs/2 even without this option.
- `-ARCH=<nch>` — number of antenna-array channels.
- `-ARRAY` — output one RTCM3 obs stream per array element (station ID = CH).
- `-SCALE=<scale>` — raw-to-IF LUT scale for INT8 / CS8 / CS16 formats (0 enables AGC; default 1.0).
- `-TRACE[=<level>]` — enable debug trace logging at `<level>`.
- `-FAST_SRCH` — enable fast acquisition (the `fast_acq` key of `sdr_rcv_setopt()` enables it as well). The receiver predicts initial Doppler from `.pocket_navdata.csv` navigation/almanac data plus the last FIX position and receiver clock drift rate, at the receiver time. Until the receiver sync mode becomes coarse (the epoch time set by the TOW/WN of a received signal) the signals are acquired by the full search; the fine mode follows the first PVT fix. The last FIX position is inherited regardless of its timestamp.
- `-GAIN=<dB>` / `-BW=<MHz>` — SoapySDR gain and analog bandwidth options passed through `sdr_rcv_open_sdev()`.
- `-E5AB_OFF=<ns>` — Galileo E5AltBOC code offset correction.
- `-BUMP_K=<value>` — BOC bump-jump discriminator parameter.
- `-SENS=<name>[,<name>...]` — sensors of the RF channels logged as `$SENS,<time>,<rfch>,<name>,<value>` every 10 s (default: `temp`).

API Functions

```
sdr_rcv_t *sdr_rcv_new(const char **sigs, const int *prns, int n, int fmt, int nrfch, double fs, const double *fo, const int *IQ, const int *bits, const char *opt)
```

- **Description**: Allocate and initialize a receiver with `n` (signal, PRN) pairs, IF data format, and per-RF-CH parameters.
- **Arguments**:
  - sigs: array of signal name strings (length `n`)
  - prns: array of PRN numbers (length `n`)
  - n: number of (sig, prn) pairs
  - fmt: IF data format (`SDR_FMT_*`)
  - nrfch: number of RF CHs (≤ 0: as defined by `fmt`)
  - fs: sampling rate (sps)
  - fo: per-RF-CH LO frequencies (length ≥ number of RF CHs)
  - IQ: per-RF-CH sampling type (1=I, 2=IQ)
  - bits: per-RF-CH sample bit width
  - opt: receiver options string
- **Return**: receiver pointer (NULL: error)
- **Notes**:
  - Sets up `sdr_rfch_t[]`, IF buffers, channel threads, PVT, and (if `-ARCH` ≥ 1) `sdr_array_t`. Use `sdr_rcv_open_*` for the common combined-allocate-and-start path.
  - A single-RF-CH format (`CS8` or `CS16`) with `nrfch` ≥ 2 means the raw IF data carries the RF CHs as channel-planar blocks of a processing cycle, while a multi-CH format (`RAW8` / `RAW16` / `RAW16I` / `RAW32`) carries them interleaved in a sample.

`void sdr_rcv_free(sdr_rcv_t *rcv)`

- **Description**: Free a receiver including all owned channels, buffers, PVT, and array state.

`int sdr_rcv_start(sdr_rcv_t *rcv, int dev, void *dp, const int *types, const char **paths)`

- **Description**: Attach a device source (`SDR_DEV_*`) and start the IF data and channel threads.
- **Arguments**:
  - rcv: receiver
  - dev: device type
  - dp: device pointer (matching `dev`: `sdr_dev_t*`, `sdr_sdev_t*`, `FILE*`, or `stream_t*`)
  - types: `SDR_MAX_STR` (= 8)-element array of output stream types (`SDR_STR_NMEA`, `SDR_STR_RTCM3`, `SDR_STR_LOG`, `SDR_STR_IFDATA`; `SDR_STR_NONE` disables a slot)
  - paths: `SDR_MAX_STR`-element array of output stream paths (NULL/"" disables a slot)
- **Return**: 1 on success, 0 on error
- **Notes**:
  - Any type can be assigned to any slot and the same type may appear in multiple slots (e.g., NMEA to a file and a TCP server simultaneously). Messages are encoded once and written to every open stream of the type. `SDR_STR_IFDATA` slots are enabled only for

`SDR_DEV_USB` / `SDR_DEV_SOAPY` inputs.

`void sdr_rcv_stop(sdr_rcv_t *rcv)`

- **Description**: Stop all threads (does not free `rcv`).

`sdr_rcv_t *sdr_rcv_open_dev(const char **sigs, int *prns, int n, int bus, int port, const char *conf_file, const int *types, const char **paths, const char *opt)`

- **Description**: Open a Pocket SDR FE USB device and start a receiver.
- **Arguments**:
  - bus / port: USB bus / port (-1 for any)
  - conf_file: optional MAX2771 / FX register file ("": skip)
  - other arguments as in `sdr_rcv_new()`
- **Return**: receiver pointer (NULL: error)

`sdr_rcv_t *sdr_rcv_open_sdev(const char **sigs, int *prns, int n, const char *driver, int fmt, double rate, const double *fo, const int *types, const char **paths, const char *opt)`

- **Description**: Open one or more SoapySDR devices and start a receiver. The number of RF CHs of the receiver is the total number of RF CHs opened.
- **Arguments**:
  - driver: SoapySDR driver or device specs (see `sdr_sdev_open()`)
  - fo: center frequency for each RF CH (Hz) (fo[i] ≤ 0: same as fo[0])
  - other arguments as in `sdr_rcv_new()`
- **Return**: receiver pointer (NULL: error)

`sdr_rcv_t *sdr_rcv_open_file(const char **sigs, int *prns, int n, int fmt, double fs, const double *fo, const int *IQ, const int *bits, double toff, double tscale, const char *file, const int *types, const char **paths, const char *opt)`

- **Description**: Open an IF data file and start a receiver replaying it.
- **Arguments**:
  - toff: file offset to start from (s)
  - tscale: replay time scale (1.0 = real-time; 0 = fastest)
  - file: IF data file path. If a `<file>.tag` exists, `fmt` / `fs` / `fo` / `IQ` / `bits` are overridden by tag values.
- **Return**: receiver pointer (NULL: error)

`void sdr_rcv_close(sdr_rcv_t *rcv)`

- **Description**: Stop the receiver, close the underlying device, and free all resources.

`void sdr_rcv_setopt(const char *opt, double value)`

- **Description**: Set a global library option. Recognized keys are: `epoch`, `lag_epoch`, `el_mask`, `sp_corr`, `t_acq`, `t_acq_ext`, `t_coh`, `t_dll`, `b_dll`, `b_pll`, `b_fll_w`, `b_fll_n`, `max_dop`, `thres_cn0_l`, `thres_cn0_ext`, `thres_cn0_u`, `thres_pli`, `lost_th`, `bump_jump`, `max_acq`, and `fast_acq`.
- **Notes**:
  - `t_acq_ext` controls assisted-acquisition integration; `t_coh` requests the coherent PLL update interval for synchronized pilot signals; `thres_cn0_ext` is the assisted-acquisition threshold before applying the tracking-loss-plus-1-dB floor.
  - `t_acq_ext`, `t_coh`, and `thres_cn0_ext` accept only values greater than zero. An invalid or unknown key prints an error and leaves the current value unchanged.

`int sdr_rcv_rcv_stat(sdr_rcv_t *rcv, char *buff, int size)`

- **Description**: Format receiver-level statistics (time, fmt/CH counts, LO/fs, sampling, IF rate, buffer usage, ...) into `buff`.
- **Return**: number of bytes written

`void sdr_rcv_str_stat(sdr_rcv_t *rcv, int *stat)`

- **Description**: Get per-output-stream connection state into `stat[0..SDR_MAX_STR-1]` (-1: error, 0: closed, 1: waiting, 2: connected, 3: active), indexed by stream slot.

`int sdr_rcv_write_str(sdr_rcv_t *rcv, int type, uint8_t *data, int size)`

- **Description**: Write `data` to every open output stream whose type is `type` (`SDR_STR_*`).
- **Return**: number of bytes written to the first stream of the type (0 if none)

`int sdr_rcv_sat_stat(sdr_rcv_t *rcv, const char *sat, char *buff, int size)`

- **Description**: Format per-satellite status text for `sat` (e.g., "G05") into `buff`.
- **Return**: number of bytes written

`int sdr_rcv_ch_stat(sdr_rcv_t *rcv, const char *sys, int all, double min_lock, int rfch, int opt, char *buff, int size)`

- **Description**: Dump receiver-channel status table.
- **Arguments**:
  - sys: filter by system ("ALL", "GPS", "GLONASS", ...)

- all: include idle channels
- min_lock: minimum lock time (s) to include
- rfch: filter by RF CH (0 = all)
- opt: option flags (reserved)
- buff/size: output buffer

- **Return**: number of bytes written

`void sdr_rcv_sel_ch(sdr_rcv_t *rcv, int ch, double width)`

- **Description**: Select the channel currently shown in correlator views (`ch` 1-indexed, 0 = none) and configure its extra-correlator span (`width` seconds, total). Internally drives `sdr_ch_set_corr()` .

`int sdr_rcv_corr_stat(sdr_rcv_t *rcv, int ch, double *stat, double *pos, sdr_cpx_t *C, double *P, double *I)`

- **Description**: Snapshot correlator state for channel `ch` (1-indexed). `stat` , `pos` , `C` , `P` , and `I` follow the same layout as `sdr_ch_corr_stat()` . Returns the number of correlator positions on success, 0 for an invalid/idle channel.

`int sdr_rcv_corr_hist(sdr_rcv_t *rcv, int ch, double tspan, double *stat, sdr_cpx_t *P)`

- **Description**: P-correlator history for channel `ch` over the past `tspan` s.
- **Return**: number of samples returned

`int sdr_rcv_rfch_stat(sdr_rcv_t *rcv, int ch, double *stat)`

- **Description**: Snapshot per-RF-CH (or per-array-CH) status into `stat[0..7]` = {dev, fmt, fs, fo, IQ, bits, std-dev, rtoc-flag}. `ch` is 1-indexed (1..nrfch+narch).
- **Return**: 1 on success, 0 on error

`int sdr_rcv_rfch_sensor(sdr_rcv_t *rcv, int ch, const char *name, double *value)`

- **Description**: Read a sensor of a RF channel of the SDR device (see `sdr_sdev_sensor()`). Pocket SDR FE, file and stream inputs have no sensors.
- **Arguments**:
  - ch: RF channel number (1 to nrfch)
  - name: sensor name as defined by the device (`temp` , `rssi` , `lo_locked` , `ref_locked` , ...)
  - value: sensor value (a boolean sensor as 1 or 0)
- **Return**: 1 on success, 0 if no such sensor

`int sdr_rcv_rfch_psd(sdr_rcv_t *rcv, int ch, double tave, int N, float *psd)`

- **Description**: Compute PSD over the latest `tave` seconds with FFT length `N`.
- **Return**: number of PSD bins written

`int sdr_rcv_rfch_hist(sdr_rcv_t *rcv, int ch, double tave, int *val, double *hist1, double *hist2)`

- **Description**: I/Q sample histograms over the latest `tave` seconds. Outputs share a common bin-value array `val[]` and per-component frequencies `hist1[]` (I), `hist2[]` (Q).
- **Return**: number of histogram bins

`int sdr_rcv_pvt_sol(sdr_rcv_t *rcv, char *buff, int size)`

- **Description**: Format the latest PVT solution as a single-line text string.
- **Return**: number of bytes written

`int sdr_rcv_get_gain(sdr_rcv_t *rcv, int ch)`

- **Description**: USB device only — get RF CH `ch` PGA gain (dB). 0 / negative on error.

`int sdr_rcv_set_gain(sdr_rcv_t *rcv, int ch, int gain)`

- **Description**: USB device only — set RF CH `ch` PGA gain (dB).
- **Return**: 1 on success, 0 on error

`int sdr_rcv_get_filt(sdr_rcv_t *rcv, int ch, double *bw, double *freq, int *order)`

- **Description**: USB device only — read MAX2771 IF filter parameters of RF CH `ch`.
- **Return**: 1 on success, 0 on error

`int sdr_rcv_set_filt(sdr_rcv_t *rcv, int ch, double bw, double freq, int order)`

- **Description**: USB device only — set MAX2771 IF filter parameters of RF CH `ch`.
- **Return**: 1 on success, 0 on error

`int sdr_rcv_array_ant_pos(sdr_rcv_t *rcv, const double *ant_pos, const int *ant_ena)`

- **Description**: Set per-RF-CH antenna body-frame positions and enable flags. After the state update, all active array CH beams are atomically refreshed with the latest geometry (current `az` / `el` preserved).
- **Arguments**:

    - rcv: receiver (must have `array != NULL`)
    - ant_pos: `nrfch*3` doubles (row-major), or NULL to leave positions unchanged
    - ant_ena: `nrfch` enable flags (`ant_ena[0]` must be 1), or NULL to leave flags unchanged
- **Return**: 1 on success, 0 on error or when `ant_ena[0] == 0`

`int sdr_rcv_array_run(sdr_rcv_t *rcv, int run)`

- **Description**: Drive the calibration EKF run flag. After the update, all active array CH beams are atomically refreshed with the latest state.
- **Arguments**:
    - rcv: receiver (must have `array != NULL`)
    - run: 0 stop, 1 start fresh (resets EKF state), 2 clear
- **Return**: 1 on success, 0 on error
- **Notes**:
    - To inject saved calibration values, use `sdr_rcv_array_load()`.

`int sdr_rcv_array_set_mode(sdr_rcv_t *rcv, int mode)`

- **Description**: Receiver-level wrapper for `sdr_array_set_mode()`. Selects what the calibration EKF estimates (`SDR_CALIB_BOTH` / `SDR_CALIB_BIAS` / `SDR_CALIB_RPY`).
- **Return**: 1 on success, 0 on error

`int sdr_rcv_array_save(sdr_rcv_t *rcv, const char *file)`

- **Description**: Snapshot the current calibration state and write it to `file`. Returns 0 (no write) when the EKF has not produced any data yet.
- **Return**: 1 on success, 0 on error / nothing to save

`int sdr_rcv_array_load(sdr_rcv_t *rcv, const char *file)`

- **Description**: Load attitude and per-CH biases from `file` (format produced by `sdr_rcv_array_save()`) and inject them into the EKF state. Refreshes all active beams afterwards.
- **Return**: 1 on success, 0 on error

`int sdr_rcv_array_stat(sdr_rcv_t *rcv, double *rpy, double *bias, double *rms, int *nep)`

- **Description**: Read current array calibration state. Outputs are filled when the array exists; `bias[0]` is always 0.
- **Return**: current calibration run flag (1 = running, 0 = stopped); 0 also when `array == NULL`.

`int sdr_rcv_array_set_beam(sdr_rcv_t *rcv, int ach, double az, double el)`

- **Description**: Set beam direction for array CH `ach` (absolute index `nrfch ≤ ach < nrfch+narch`). Recomputes weights from the current calibration state and antenna geometry.
- **Return**: 1 on success, 0 on error

`int sdr_rcv_array_get_beam(sdr_rcv_t *rcv, int ach, double *az, double *el)`

- **Description**: Get the most recently set beam direction for array CH `ach`.
- **Return**: 1 on success, 0 on error

`void sdr_rcv_array_calib(sdr_rcv_t *rcv, const obsd_t *obs, int nobs, const nav_t *nav, const double *rr)`

- **Description**: Per-epoch driver invoked by `sdr_pvt_udsol()` after each PVT update. Holds `rcv->mtx` while calling `sdr_array_calib()`.

sdr_web.c - Web UI Server Functions

Overview

HTTP and WebSocket server for the Web UI of the `pocket_web` AP, implemented directly on BSD sockets / winsock2 with no external dependency (own SHA-1 and Base64 for the WebSocket handshake). A single server thread runs a `select()` loop on a 10 ms tick that also drives the topic scheduler, so no thread is created per client.

Static Web UI files are served over HTTP from the document root; commands and monitor data are exchanged over one WebSocket per client. Each client subscribes to topics with its own cycle, and the server pushes the latest data only (no queueing) as JSON text frames or, for the PSD, correlator and correlator history, as little-endian binary frames. See `doc/design_web_ui.md` for the wire protocol.

The server also owns the receiver lifecycle. It keeps a `sdr_web_cfg_t` configuration, creates and closes the receiver on the Start / Stop commands, and saves the configuration and the system options to the settings file when the receiver stops and when the server stops.

API Functions

`sdr_web_t *sdr_web_start(sdr_rcv_t *rcv, const char *addr, int port, const char *html_dir)`

- **Description**: Start the Web UI server and its server thread.
- **Arguments**:
  - rcv: SDR receiver to monitor (NULL: start with no receiver)
  - addr: bind address ("": 127.0.0.1, loopback only)
  - port: TCP port
  - html_dir: document root of the Web UI files ("": `<exe_dir>/../html`)
- **Return**: Web UI server (NULL: error)

`void sdr_web_init_cfg(sdr_web_cfg_t *cfg)`

- **Description**: Set the default receiver configuration, as used on the first start without a settings file.

`void sdr_web_set_cfg(sdr_web_t *web, const sdr_web_cfg_t *cfg, const char *file)`

- **Description**: Set the receiver configuration and enable the receiver lifecycle control (Start / Stop and the configuration commands) on the Web UI. Without this call the server is monitor-only.
- **Arguments**:
  - web: Web UI server (NULL: no operation)

- cfg: receiver configuration
- file: settings file to save and restore ("": no save)

`int sdr_web_load_cfg(sdr_web_t *web)`

- **Description**: Restore the receiver configuration and the system options from the settings file set by `sdr_web_set_cfg()`.
- **Return**: 1 if restored, 0 if there is no settings file

`int sdr_web_start_rcv(sdr_web_t *web)`

- **Description**: Start the SDR receiver with the current configuration, as the Start command of the Web UI does. The current receiver, if any, is closed first.
- **Return**: 1 if started, 0 on error or if the receiver is already running

`sdr_rcv_t *sdr_web_stop(sdr_web_t *web)`

- **Description**: Stop the Web UI server, save the settings and free all resources. The current receiver is not closed; it is returned to the caller to be closed with `sdr_rcv_close()`.
- **Return**: current SDR receiver (NULL: none)